

Loops

Introduction to Computer Programming

Chandra Gummaluru

`chandra-gummaluru.github.io`

A handwritten signature in black ink, appearing to read 'Gummaluru'.

Often we want to do the same thing many times. A **loop** tells Python to repeat a block of code, instead of us writing it out.

without a loop

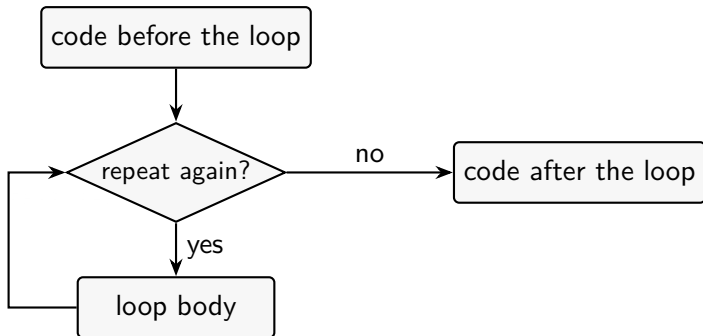
```
print(1)
print(2)
print(3)
print(4)
print(5)
```

with a loop

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

to count to 1000, the left grows to 1000 lines; the right changes one number

Every loop has a **body**: the indented block it repeats. Each run through the body is one **pass**.



the loop starts by asking whether it should “repeat again?”

There are two types of loops. They differ in how they decide whether to repeat.

for loop

once for each element

```
s = 'hi'  
for ch in s:  
    print(ch)
```

stops when it runs out of elements

while loop

as long as a condition is True

```
x = 1  
while x < 100:  
    x = x * 2
```

stops when the condition is False

How many times does the body of this loop run?

```
1 i = 10
2 while i > 0:
3     i = i - 3
```

How many times does the following loop execute?

```
1 for ch in 'chandra':  
2     print(ch)
```

How many times does the following loop execute?

```
1 s = 'chandra'
2 for ch in s:
3     s = s[:-1]
4     print(s)
```

Loops are often used to build up an answer as they go, in a variable called the **accumulator**.

(1) **before the loop**

set the accumulator to an initial value

(2) **during the loop**

update the accumulator during each pass of the loop

(3) **after the loop**

the accumulator holds the answer

Depending on what you are building, the starting value for the accumulator, and how it is updated will change.

accumulator	initial value	update
string	' '	<code>s_new = s_new + ch</code>
number	0/1	<code>total = total + i</code>
True/False	False	<code>found = True</code>

for loops

A **for loop** runs its body once for each element in an object. Each pass, the loop variable holds the next element.

```
s = 'cat'  
for ch in s:  
    print(ch)
```

ch	printed
'c'	c
'a'	a
't'	t

when there are no elements left, the loop ends

To build a **string**, start with '' and add to it on each pass.

```
doubled = ''  
s = 'cat'  
for ch in s:  
    doubled = doubled + ch + ch  
print(doubled)
```

ch	doubled
—	''
'c'	'cc'
'a'	'ccaa'
't'	'ccaatt'

What does this print?

```
1 s1 = 'Avatar Aang'
2 s2 = ''
3 for ch in s:
4     if ch.isupper():
5         s2 = s2 + ch.lower()
6     else:
7         s2 = s2 + ch.upper()
8 print(s2)
```

A Avatar Aang

B aVAtAR aANg

C AvAtAr AaNg

D An error

vowels.py

Program

```
1 def only_vowels(s: str) -> str:
2     """Return a string containing only the vowels from s.
3
4     The vowels are 'a', 'e', 'i', 'o', and 'u'.
5     The vowels should appear in the same order as they
6     appear in s.
7
8     Preconditions:
9         s contains only lowercase letters
10
11     Examples:
12         >>> only_vowels('computer')
13         'oue'
14         >>> only_vowels('hello')
15         'eo'
16         >>> only_vowels('python')
17         'o'
18         >>> only_vowels('xyz')
19         ''
```

```
20      ""  
21      # Write your code here  
22  
23  
24  print(only_vowels('computer'))
```

while loops

A **while loop** runs its body as long as a condition is True. The condition is checked at the start of every pass.

```
i = 0
while i < 3:
    print(i)
    i = i + 1
```

i	i < 3	printed
0	True	0
1	True	1
2	True	2
3	False	—

something in the body has to move the loop toward stopping; without `i = i + 1`, it runs forever

To build a **number**, start at 0/1 and add to it on each pass.

```
total = 0
i = 1
while i <= 4:
    total = total + i
    i = i + 1
print(total)
```

i	total
—	0
1	1
2	3
3	6
4	10

What does this print?

```
1 n = 1000
2 count = 0
3 while n > 1:
4     n = n // 10
5     count = count + 1
6 print(count)
```

digits.py

Program

```
1 def digit_sum(n: int) -> int:
2     """Return the sum of the digits in n.
3
4     Preconditions:
5         n >= 0
6
7     Examples:
8         >>> digit_sum(4096)
9         19
10        >>> digit_sum(123)
11        6
12        >>> digit_sum(7)
13        7
14        >>> digit_sum(0)
15        0
16        """
17        # Write your code here
18
19
```

```
20 print(digit_sum(4096))
```

To check whether a condition holds for **any** value, start at False, and switch to True as soon as you find one for which it does.

```
## is any of 1..5 a multiple of 4?
found = False
i = 1
while i <= 5:
    if i % 4 == 0:
        found = True
    i = i + 1
print(found)
```

i	i % 4	found
—	—	False
1	1	False
2	2	False
3	3	False
4	0	True
5	1	True

there is no else: once found is True, nothing sets it back

9 has a divisor (3), so it should print `True` . Does it?

```
1 n = 9
2 found = False
3 d = 2
4 while d < n:
5     if n % d == 0:
6         found = True
7     else:
8         found = False
9     d = d + 1
10 print(found)
```

☐ A True

☐ B False

primes.py

Program

```
1 def is_prime(n: int) -> bool:
2     """Return whether n is a prime number.
3
4     A prime number is a positive integer greater than 1
5     that has exactly two positive divisors: 1 and itself.
6
7     Preconditions:
8         n >= 2
9
10    Examples:
11        >>> is_prime(2)
12        True
13        >>> is_prime(7)
14        True
15        >>> is_prime(9)
16        False
17        >>> is_prime(12)
18        False
19    """
```



```
20     # Write your code here
21
22
23 print(is_prime(9))
```

To check whether a condition holds for **every** value, start at True and switch to False as soon as you find one for which it doesn't.

```
## is every one of 1..5 odd?  
all_odd = True  
i = 1  
while i <= 5:  
    if i % 2 == 0:  
        all_odd = False  
    i = i + 1  
print(all_odd)
```

i	i % 2	all_odd
—	—	True
1	1	True
2	0	False
3	1	False
4	0	False
5	1	False

one even number is enough to make the answer False; the rest can't undo it

2 and 4 are even, so the answer should be `False` . Does this print `False` ?

```
1 all_odd = True
2 i = 1
3 while i <= 5:
4     if i % 2 == 0:
5         all_odd = False
6     else:
7         all_odd = True
8     i = i + 1
9 print(all_odd)
```

☐ A True

☐ B False

What does this print?

```
1 s = 'banana'
2 count = 0
3 for ch in s:
4     if ch == 'a':
5         count = count + 1
6 print(count)
```

Does this print `True` ?

```
1 s = 'Hello'
2 all_lower = True
3 for ch in s:
4     if ch.isupper():
5         all_lower = False
6 print(all_lower)
```

☐ A True

☐ B False

What does this print?

```
1 s = 'a1b22c'
2 total = 0
3 for ch in s:
4     if ch.isdigit():
5         total = total + int(ch)
6 print(total)
```

What does this print?

```
1 s = 'computer'
2 result = ''
3 i = 0
4 while i < len(s):
5     result = result + s[i]
6     i = i + 2
7 print(result)
```

A

cmue

B

optr

C

cmpt

D

computer

What does this print?

```
1 n = 5
2 s = ''
3 while n > 0:
4     if n % 2 == 1:
5         s = s + str(n)
6     n = n - 1
7 print(s)
```

A

135

B

531

C

54321

D

9

What does this print?

```
1 s = 'ab3cd7'
2 i = 0
3 found = False
4 while i < len(s) and not found:
5     if s[i].isdigit():
6         found = True
7     i = i + 1
8 print(i)
```

What does this print?

```
1 n = 6
2 bits = ''
3 while n > 0:
4     bits = str(n % 2) + bits
5     n = n // 2
6 print(bits)
```

A 011

B 110

C 6

D 0110